

COMPUTE!'s Gazette

Future Computing: Neural Networks

A three-part series by Kevin E. Martin
January–March 1990 • Commodore 64 / 128

Part 1 — January 1990 (Issue 79)

Introduction to neural computing, neurons, network classes,
and the linear associator.

Part 2 — February 1990 (Issue 80)

Back-propagation, three-layer BP networks, XOR examples,
and the BP machine-language program with BASIC interface.

Part 3 — March 1990 (Issue 81)

Competitive learning / self-organizing networks and the
Dipole ML program with BASIC interface.

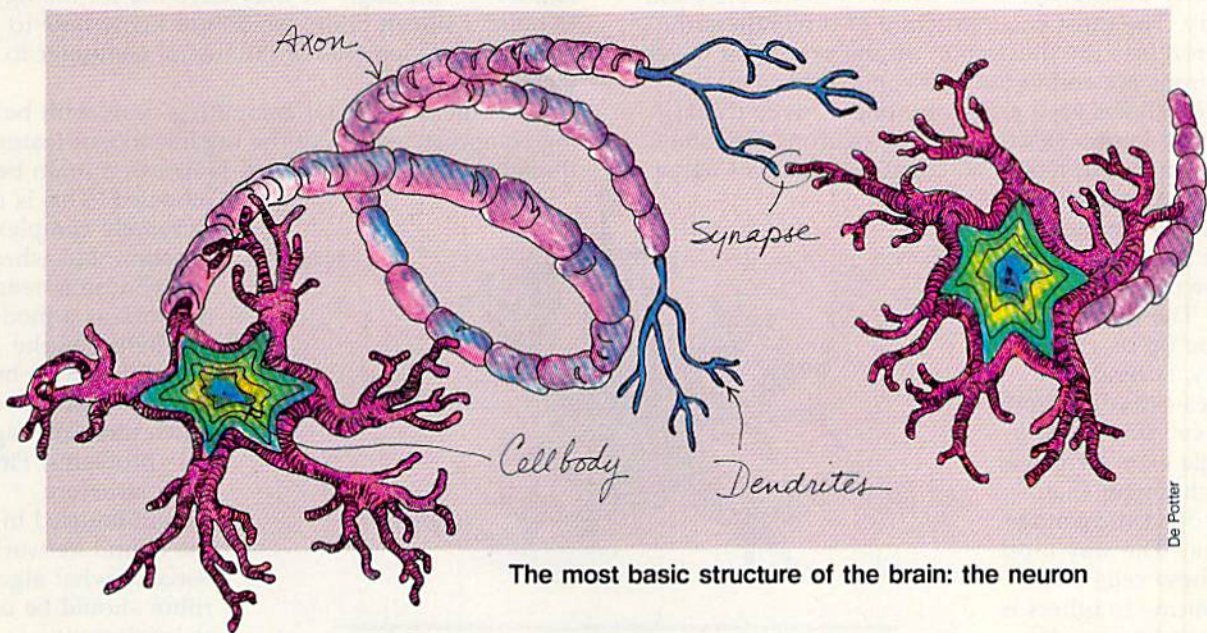
Source scans: Internet Archive identifiers

1990-01-computegazette, 1990-02-computegazette,
1990-03-computegazette

This compilation extracts the article pages (including type-in
listings) from the original magazine scans. Searchable text is
the Internet Archive OCR layer carried through from those PDFs.

Compiled September 16, 2026.

Future Computing



The most basic structure of the brain: the neuron

Neural Networks

Part 1

Kevin E. Martin

This month we debut the first in a three-part series on neural networks, probably the most promising line of AI (Artificial Intelligence) research today. Author Kevin E. Martin, currently working on graphics development at Data General, is best known to Gazette readers as the author of several popular utilities and applications, including SpeedCalc, Screen-80, MiniFiler, and X-BASIC.

Someday, computers will be able to speak, hear, see, and maybe even think. Of course, we're a long way from that day. Today, researchers are trying to discover the workings of the human brain by building computer models similar to structures that are thought to exist in the brain. These structures are called *neural networks*. The search has spawned a new science, neural computing, and we're just now beginning to see some significant advances within this field.

To understand tomorrow's computers, you'll need to understand what we know about the brain today. The most basic structure of the brain is the neuron (see the illustration on the preceding page). Neurons are connected in a complex structure which allows each and every one of us to think. No one knows exactly how the neurons are interconnected, but nearly all cognitive scientists agree that the power of the brain lies in the way that the neurons are interconnected.

The neuron is made up of a cell body, several dendrites extruding from the cell body, and a single axon (which is much longer than the dendritic connections). The way most of these cells communicate to others is by sending an electric pulse down the axon and transmitting it to other cells that have dendrites near the active axon. The region where the chemical interaction takes place is called a *synapse*.

When one cell transmits its pulse across the synapse to the another cell's dendrites, an electric potential builds up in the second cell. Once this reaches a certain threshold, a pulse is sent down the second cell's axon to yet another cell's dendrites. This is the process by which the neurons in the brain communicate.

Neural communication is fascinating, but we still haven't discussed the brain's most amazing quality—its ability to learn. In 1949, Donald O. Hebb proposed a technique by which the brain could learn, and it is now commonly known as the *Hebb synapse*. It can be described as a modification of the strengths of the connections between two cells, but Hebb himself described it best: "When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it,

some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

How close are we to building an electronic brain? Today's technology does allow us to build computers that have the size and complexity of the brain. Computers are made from silicon chips which function at nearly 100,000 times the speed of neurons. You might conclude that the computer would be that much faster and more powerful than a human, but even the fastest digital computer of today cannot tell the difference between an arm and a leg. Although we may have the technology to build a silicon brain, we do not know how to organize the elements of the neural computer to yield intelligence.

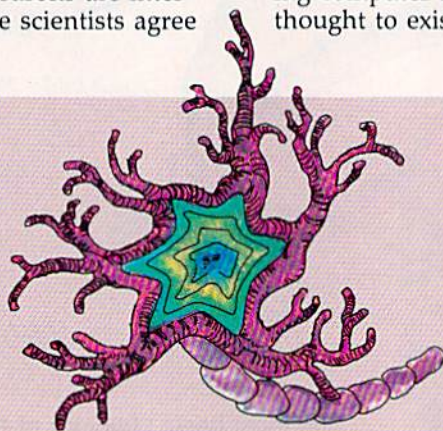
The field of neural computing deals with building computer models of the organizational features thought to exist in the brain. How can a brain be

modeled? This is an extremely complex question, but when you choose a neural network as a model for the brain, the question can be broken down into two much more manageable problems: First, what structure should be used in the neural network? Second, what algorithm should be used to implement learning?

Many types of neural network models have been developed over the past several years. Most fit into three broad categories: associator networks, optimizing

networks, and self-organizing networks. The associator will learn to associate an input and an output pattern so that when you present the network with the input pattern, it will return the correct output pattern. One example of an associator is discussed in the accompanying sidebar, "The Linear Associator."

Optimizing neural networks are used in problems in which an optimum solution is desired, and no easy algorithm exists. This neural network architecture was first explored by physicist J. J. Hopfield in 1982. This type of neural network deals with ideas taken from statistical physics, simulated annealing, and thermodynamics. The basic idea is to minimize the energy in the system. First, introduce a parameter into the neural network for temperature and start the network out at a high temperature. This causes the *processing elements* (PEs) to



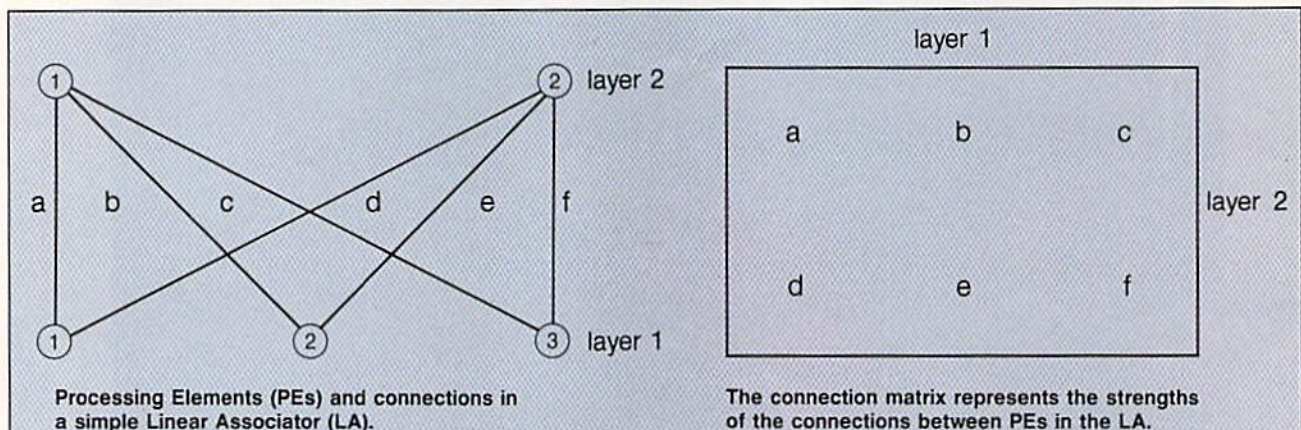
Even the fastest digital computer of today cannot tell the difference between an arm and a leg.

fire rapidly. Then, slowly cool the network down until it settles into a solution.

Self-organizing neural networks are much closer to the organizational principle thought to exist in the human brain. Our brain begins in a random state, and as we learn more and more about the world and its complex interrelationships, our brain begins to organize itself in a very structured way. No two brains are exactly alike, but generally they have the same organization. The self-organizing neural networks start from a random state and slowly organize themselves into a very structured pattern.

The LA neural network described in the sidebar learns to associate patterns of inputs and outputs, but there are limitations. Only certain inputs are learnable by the LA. Unfortunately, the number of these inputs it can learn is limited to the number of PEs in the input layer. So, even though you may have five PEs in the first layer and you can present 32 different input patterns to these five PEs, you still can make the network learn only five patterns.

A solution to the problem is found in another type of associating neural network, *back propagation*. This architecture is the most commonly used neural network today, and we'll discuss it in Part 2. **G**



The Linear Associator

Let's take a quick look at the linear associator (LA), a simple neural network used to associate inputs and outputs through a single layer of connections.

Although most neural networks have very different structures, some similarities can be found:

- A set of processing elements (PEs)
- A connection matrix containing the weights of each connection between any two PEs
- The level of activation
- The learning and retrieval phases
- An input vector
- An output vector

In a linear associator, each processing element (PE)—represented in the diagram by a circle with a number inside of it—corresponds to the body of the neuron, or in some cases, each PE corresponds to a group of neurons collectively working together. Each PE represents some concept in the real world. Maybe a word, a letter, or even a single pixel in a grid where pictures of letters are presented. The PEs are divided into separate groups, or layers, and they are numbered in order by layers. The lines connecting the PEs of different layers correspond to the axon connections between neurons, and these connections are stored in the connection matrix.

When two PEs from different layers are connected, the strength, or weight, of the connection between them is stored in the connection matrix. The connection matrix is used to determine how to propagate signals between layers. The signal starts in the layer listed on top of the connection matrix and propagates to the layer listed along the side of the matrix. In this LA, the signal starts in layer 1 and goes to layer 2. Normally, the signal moves from a layer with a lower number to a layer with a higher number. This is not the case in all neural networks, but it generally holds true.

The weight of a connection is normally denoted by a number. If the connection is excitatory (that is, if the PE in the first layer excites the PE in the second layer), the weight of the connection is positive. If the connection is inhibitory (that is, if the PE in the first layer inhibits the PE in the second layer), the weight is negative. In the LA (and most other neural networks), you need to have both excitatory and inhibitory connections between PEs.

Every PE can be on, off, or somewhere in between. This is known as its *activation level*. The range of activation is usually between 0 and 1 or between -1 and 1. The LA uses activations between 0 and 1, where 0 is completely off and 1 is completely on. A PE is said to be active when its level of activation is near 1. Then, if the connection between it and another PE in the next layer is excitatory, the PE in the next layer becomes more active (that is, its activation level increases). The opposite happens when the connection is inhibitory.

The purpose of a neural network is to transform inputs to outputs. It does this in two distinct phases: learning and retrieval. During the learning phase in the LA, you present input and output vectors, and the network learns to associate the two patterns. During the retrieval phase, you present an input vector and the network gives you the output vector it has learned.

To teach our LA the correct patterns, we need to have a learning algorithm which will modify the weights in the connection matrix so that during the retrieval phase (as above), the correct output vector will be given. The process of neural network learning is probably the single most important feature of the network model. Without a learning algorithm, the networks could do only what they were designed to do when they were created. The most important feature of the learning algorithm to remember is that it works by modifying the weights in the connection matrix.

Future Computing Neural Networks

Part 2

Kevin E. Martin

In this month's installment, we'll look at some programs you can use to begin to understand this exciting branch of neural networks.

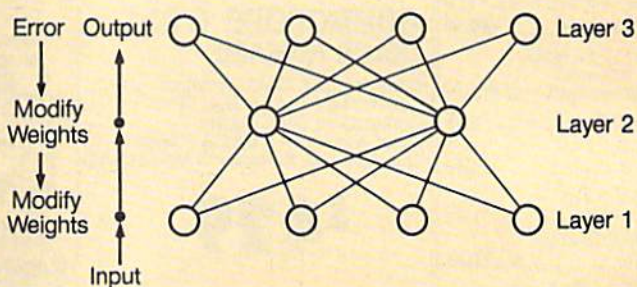
Last month we introduced the topic of neural networks with a general discussion, and we took a closeup look at the linear associator (LA) neural network. The LA is able to learn to associate different input and output patterns, but there are limitations. Only certain input patterns are learnable.

The number of input patterns the LA can learn is equal to the number of processing elements (PEs) in the first layer of the network. For example, take a network that contains ten PEs in the first layer. You can present 1024 different patterns to the input layer, but the network can learn only 10 of them exactly. If you try to force the network to learn more, the patterns it previously learned will become contaminated.

This limitation makes the back propagation (BP) neural network a better choice for practical applications. In fact, back propagation is the most commonly used neural network architecture used in the business world today. To get you started in your explorations of neural networks, I've included a machine language program with an interface to BASIC. The program, BP, implements the back propagation algorithm. Also listed are two examples to illustrate the power of this algorithm.

To overcome the limitations of the linear associator, back propagation introduces another layer in the middle of the network (see Figure 1). The middle layer allows the network to develop an internal representation of the problem it is assigned to solve. This internal representation is developed entirely within the network. As you can see in Figure 1, layer 2 receives input from layer 1

Figure 1



The layout of a back propagation network.

and outputs its results to layer 3. This means that it doesn't receive any inputs or outputs directly from the outside world, and therefore a learning algorithm that can develop this internal representation is required. The internal representation codifies the inputs into many different features. These features will be discussed in greater depth in the exclusive-OR example below.

The example programs are implemented using a three-layer BP neural network (so called because there are three layers of PEs in this implementation). For a back propagation network, the minimum number of lay-

ers is 3, but in more complex situations, the problem may require many more.

The linear associator was described in the previous article to introduce several important characteristics of all neural network architectures. The back propagation neural network is very similar to the linear associator, but some important differences exist, namely: the number of layers, the output function, and the teaching algorithm.

The number of layers affects the organization of the matrix in only a minimal way. In the LA, we had a single connection matrix for the network representing the connections between the PEs in the first and second layers. For the BP network, we have three layers of PEs and two layers of connections. To accommodate this new layer of connections, we need two connection matrices: one for the connections between layers 1 and 2, and one for the connections between layers 2 and 3. Layer 1 receives the inputs from outside, and layer 3 presents the resulting outputs.

The back propagation network may seem like two linear associators placed one on top of the other, but, in the BP network, there are output functions for each PE in each layer. In the LA, the output of each PE was simply its activation level (calculated by collecting the inputs from the previous layer, multiplying them through the connection matrix, and then summing them). In the BP network, the activation level is calculated in a similar manner, but then it is used as the parameter in an output function that actually generates the PE's output.

You no doubt have heard the saying *practice makes perfect*. This phrase is true for back propagation neural networks. For these networks to learn the patterns correctly, you need to present the input and teacher patterns many times (see Figure 1).

The more times you present the input and teacher patterns, the better the system learns to associate these patterns. The total error gives you an idea of the progress your system is making in learning the patterns. As you continue to present the patterns, the total error declines until it eventually reaches a minimum. If you plot the total error against the number of trials, you'll see the error start high, slowly lessen, and eventually reach a minimum value (see Figure 2).

An excellent way to picture this system is to use the analogy of a ball rolling down a hill. Hills generally descend, but they may contain small valleys. Figure 3 illustrates this idea. As the ball rolls down the error hill, it gains momentum. If it has enough momentum to escape the small, secondary valleys on its way down, it eventually ends up at the bottom of the hill.

Hands On

BP is a back propagation simulator that allows you to experiment with your own systems. It handles all of the processing for the network; you simply define the network's characteristics, teach it the patterns you wish it to

learn, and then save the network. When you want to have the system recognize these patterns, you simply load the network back into memory and present the input patterns; the system outputs the patterns you taught it. To help you get started, I've included two classic example problems that a back propagation network can learn but that earlier networks, such as the linear associator, cannot.

BP is written in machine language, so you'll need to use MLX, the machine language entry program found elsewhere in this issue, to enter it. The MLX prompts, and the values you should enter, are as follows:

Starting address: C000
Ending address: CFFF

Once you've finished entering the data, save a copy of the program with the name BP.ML. (The example programs expect to find a file with this name.)

XOR and ENCODE are example programs that demonstrate how to use BP to simulate a back propagation neural network. Both are written in BASIC, so use *The Automatic Proofreader*, also found in this issue, to prevent typing mistakes as you enter them. Save copies of the programs to the disk containing BP.

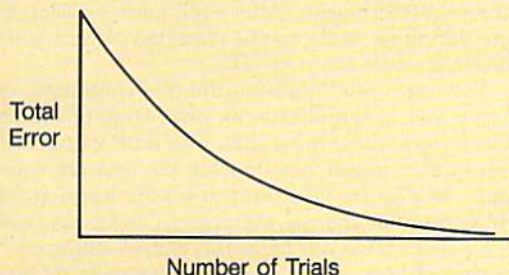
For a quick look at how BP works, load the XOR example program and type RUN. The program loads BP, sets up the network, and then begins learning the patterns. Once it has learned the patterns, the program presents each of the learned patterns to the system and displays the resulting output.

The Tools

BP has six commands for processing and managing a simple back propagation network. In the following paragraphs, you'll find descriptions of what each command does and how to use it. If you don't understand the commands at first, don't worry. You'll understand after we examine the XOR example.

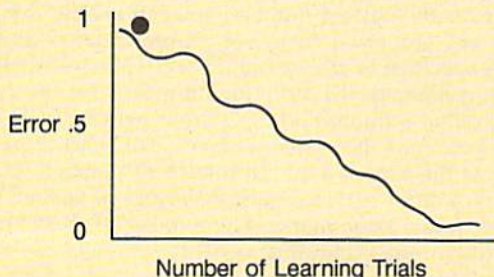
SYS 49152,fpe,spe,tpe,np,lr,momen,err Initialize the BP network. This command is used to set up a simple back propagation network. It initializes the system by creating eight BASIC variables and 11 arrays (see Table 1). These variables and arrays are used to hold the results generated by the network. Because they are BASIC variables, they are erased if you perform a NEW or CLR command, load another program, or change the existing program, so be careful. Next, *fpe*, *spe*, and *tpe* determine how many processing elements are in the first, second, and third layers of the network, respectively. Theoretically, these values can range from 1 to the maximum number of elements free memory will allow. In practice, training times for larger networks could be prohibitively long. The fourth argument, *np*, is the number of patterns that the network is to learn. This value and the number of PEs in

Figure 2



The error starts high but moves lower and eventually crosses a preset threshold.

Figure 3



The error in a neural network acts much like a ball rolling down a hill.

Future Computing

the second layer are interdependent. (If the network doesn't have enough PEs in the second layer to develop internal representations of the patterns, then the program will go into an endless loop.) The argument *lr* is the rate at which the network learns the patterns, and *momentum* is the momentum the system has as it learns a pattern. Both of these parameters will be discussed in more detail below. The last argument, *err*, is the maximum allowable error in the system. *BP* uses this value to determine when it has learned the patterns.

Table 1. Variables and Arrays Created by BP

Variable	Meaning
p1	Number of PEs in layer 1
p2	Number of PEs in layer 2
p3	Number of PEs in layer 3
np	Number of patterns for the system to learn
rate	Learning rate
momentum	Momentum of learning
epsilon	Maximum allowable error
te	Total error after each teaching trial
Array(Size)	Meaning
o2(p2)	Output from layer 2
o3(p3)	Output from layer 3
e2(p2)	Error at layer 2
e3(p3)	Error at layer 3
w1(p2,p1)	Connection weights between layers 1 and 2
w2(p3,p2)	Connection weights between layers 2 and 3
m1(p2,p1)	Momentum of the connection weights between layers 1 and 2
m2(p3,p2)	Momentum of the connection weights between layers 2 and 3
t(p3,np)	Teacher patterns
in(p1,np)	Input patterns
e(np)	Error from each pattern

SYS 49155,pat Recognize a pattern. After the system has learned the patterns you taught it, use this command to present a pattern for the network to recognize. The *pat* argument is a string of 0s and 1s that represents the pattern. It must contain the same number of 0s and 1s as there are PEs in the first layer.

SYS 49164,se Begin learning patterns. Use this command to tell *BP* to begin learning the patterns. Before you use this command, you must initialize the network with the command **SYS 49152** and present the training pairs with the command **SYS 49167**. The show-error argument, *se*, determines whether or not *BP* displays the total error after each learning trial. Set *se* to 1 to display the total error; otherwise, set it to 0.

SYS 49167,pn,ip,tp Initialize the training pairs. After you've set up your network, you must use this command to define the patterns the network is to learn. These patterns are given in pairs, called *training pairs*, that consist of an input pattern and a teacher pattern. In the command, *pn* is the pattern number. You can specify the patterns in any order you wish, but you must define all the patterns specified in the *np* parameter of the initialize *BP* network command. All of the training pairs for the system are called a *training set*. The input pattern, *ip*, is the pattern you want the system to learn. The teacher pattern, *tp*, is the expected output that corresponds to the input pattern. Both patterns must be strings of 0s and 1s enclosed in quotation marks. The number of characters in the input pattern string must equal the number of PEs in the first layer. The number of characters in the teacher pattern must correspond to the number of PEs in the third layer.

SYS 49170,filename Save network. This command saves the variables and arrays generated by *BP*. With it, you can recall a network quickly without having to wait for the system to relearn patterns. To save a network, simply type the command followed by the filename in quotation marks.

SYS 49173,filename Load network. This command loads a network saved with the save-network command above.

XOR

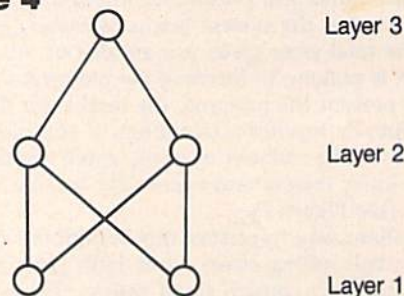
The XOR (exclusive or) function is a classic example of the type of problem the back propagation network can learn to solve. We'll go through XOR to show you how to use *BP*'s functions and to give hints on how to design your own systems.

Table 2. The Exclusive-OR Function

0	XOR	0	=	0
1	XOR	0	=	1
0	XOR	1	=	1
1	XOR	1	=	0

The XOR function operates on two input values and returns one output value (see Table 2). At the lowest level, the function combines two input bits to generate one output bit. The network represents the XOR function by having two PEs in the first layer and one PE in the third layer (see Figure 4). The number of PEs in the middle layer depends on how many patterns the system must learn. In this case, we want it to learn four patterns, so the second layer contains two PEs. (Two is the minimum number needed to learn four patterns.)

Figure 4



The layout of a BP network to solve the XOR problem.

In line 50 of *XOR*, we initialize the network with the command **SYS 49152**. The first three values following **SYS 49152** determine how many PEs are placed in the first, second, and third layers of the network (2, 2, and 1, respectively). The fourth value indicates that we want to teach the system four patterns.

The network learns the patterns by conducting learning trials. In each trial, the internal representation of a pattern is modified so that its output more closely matches the expected output. After each learning trial, the error is the difference between the expected output and the actual output from the network.

The fifth value following the **SYS** command, the learning rate, determines what percentage of the error is reduced for each learning trial. The sixth value, the momentum, also affects how quickly the network learns a pattern. In practice, you want to set the learning rate as high as possible and the momentum as low as possible. If the learning rate is too high, the system learns each pattern very quickly, but it destroys other patterns that it may have already learned. If the momentum is set too high, the system learns each pattern quickly, but it forgets the

pattern as soon as it attempts to learn the next. Your objective is to teach the network a set of patterns, not just one pattern. To achieve the best results, you want to maximize the learning rate while minimizing the momentum.

The last value following the SYS command is the total allowable error in the system. After each learning trial, the total error in the system is the sum of the errors for each training pair. When the total error becomes less than the total allowable error, the network has learned the patterns well enough. If you set this value too close to 0, BP may never stop trying to learn the patterns.

Learning Patterns

After the network has been initialized, the program initializes the training pairs. Lines 60-90 of the program present the input and teacher patterns.

Now BP is ready to run. The SYS 49164 command in line 140 tells it to start learning patterns. The 1 following the SYS command tells BP to print the total error after each learning trial. Learning all of the patterns might take quite awhile. You can break out of the program at any time by holding down the RUN/STOP key until the program completes the current trial.

After the network has learned a set of patterns, lines 170-280 display the output from the system. Line 170 presents the first pattern to the system with the command SYS 49155,"00." The pattern must be in quotation marks and should be exactly as long as the number of PEs in the first layer. After the SYS, the arrays o2 and o3 contain the output from layers 2 and 3, respectively.

Program 1. BP

```
C000:4C 18 C0 4C 4D C4 4C FE 9F
C008:C6 4C 4B CA 4C B0 CA 4C 1D
C010:04 CB 4C F7 CB 4C 63 CD B3
C018:20 FD AE 20 9E B7 8E A9 9B
C020:02 20 FD AE 20 9E B7 8E CE
C028:AA 02 20 FD AE 20 9E B7 4E
C030:8E AB 02 20 FD AE 20 9E AF
C038:B7 8E AC 02 20 FD AE A5 EA
C040:7A 8D A7 02 A5 7B 8D AB 56
C048:02 A9 8B 85 7A A9 CF 85 9E
C050:7B 20 8B B0 8D AD 02 8C C7
C058:AE 02 AD A7 02 85 7A AD AA
C060:A8 02 85 7B 20 8A AD AE 54
C068:AD 02 AC AE 02 20 D4 BB B7
C070:20 FD AE A5 7A 8D A7 02 0D
C078:A5 7B 8D AB 02 A9 8E 85 41
C080:7A A9 CF 85 7B 20 8B B0 20
C088:8D B1 02 8C B2 02 AD A7 E6
C090:02 85 7A AD A8 02 85 7B 72
C098:20 8A AD AE B1 02 AC B2 0F
C0A0:02 20 D4 BB 20 FD AE A5 7D
C0A8:7A 8D A7 02 A5 7B 8D AB BE
C0B0:02 A9 91 85 7A A9 CF 85 C7
C0B8:7B 20 8B B0 8D AF 02 8C 38
C0C0:B0 02 AD A7 02 85 7A AD 14
C0C8:A8 02 85 7B 20 8A AD AE BC
C0D0:AF 02 AC B0 02 20 D4 BB 41
C0D8:A5 7A 8D A7 02 A5 7B 8D 23
C0E0:A8 02 A9 94 85 7A A9 CF EE
C0E8:85 7B 20 8B B0 48 98 48 E8
C0F0:AD A9 02 20 3C BC 68 A8 44
C0F8:68 AA 20 D4 BB A9 97 85 E3
C100:7A A9 CF 85 7B 20 8B B0 A1
C108:48 98 48 AD AA 02 20 3C 93
C110:BC 68 AB 68 AA 20 D4 BB E2
C118:A9 9A 85 7A A9 CF 85 7B 82
C120:20 8B B0 48 98 48 AD AB 1E
C128:02 20 3C BC 68 AB 68 AA 69
C130:20 D4 BB A9 9D 85 7A A9 AC
C138:CF 85 7B 20 8B B0 48 98 BE
C140:48 AD AC 02 20 3C BC 68 DC
C148:A8 68 AA 20 D4 BB A9 FC 77
C150:85 7A A9 CF 85 7B 20 8B 4D
C158:B0 8D D2 02 8C D3 02 A9 73
C160:2E 85 7A A9 CF 85 7B AD 7F
C168:2E CF 20 81 B0 A9 A6 85 12
```

```
C170:7A A9 CF 85 7B 20 8B B0 12
C178:8D B3 02 8C B4 02 A9 01 BA
C180:20 3C BC AE B3 02 AC B4 59
C188:02 20 D4 BB A9 AC 85 7A F0
C190:A9 CF 85 7B 20 8B B0 8D 63
C198:B5 02 8C B6 02 A9 01 20 4D
C1A0:3C BC AE B5 02 AC B6 02 D4
C1A8:20 D4 BB A9 00 8D 36 03 29
C1B0:A9 B2 85 7A A9 CF 85 7B 21
C1B8:20 8B B0 8D C9 02 8D 34 C3
C1C0:03 8C CA 02 8C 35 03 A9 4B
C1C8:01 20 3C BC AC 35 03 AE 17
C1D0:34 03 20 D4 BB EE 36 03 89
C1D8:AD AC 02 CD 36 03 90 23 7D
C1E0:A9 00 8D 38 03 AD 34 03 A8
C1E8:18 69 05 8D 34 03 AD 35 8A
C1F0:03 69 00 8D 35 03 EE 38 F4
C1F8:03 AD A9 02 CD 38 03 B0 C4
C200:E4 90 C4 A9 00 8D 36 03 F4
C208:8D 38 03 A9 BA 85 7A A9 E7
C210:CF 85 7B 20 8B B0 8D BF 4A
C218:02 8D 34 03 8C C0 02 8C B0
C220:35 03 A9 01 20 3C BC 20 D1
C228:97 E0 20 E2 BA 20 0C BC 0F
C230:A9 05 20 3C BC 20 53 BB 59
C238:AC 35 03 AE 34 03 20 D4 6F
C240:BB EE 36 03 AD 34 03 18 B2
C248:69 05 8D 34 03 AD 35 03 F4
C250:69 00 8D 35 03 AD AA 02 B5
C258:CD 36 03 B0 C5 EE 38 03 1B
C260:AD A9 02 CD 38 03 90 08 3B
C268:A9 00 8D 36 03 4C 22 C2 28
C270:A9 00 8D 36 03 8D 38 03 A1
C278:A9 C2 85 7A A9 CF 85 7B EE
C280:20 8B B0 8D C1 02 8D 34 4D
C288:03 8C C2 02 8C 35 03 A9 14
C290:01 20 3C BC 20 97 E0 20 33
C298:E2 BA 20 0C BC A9 05 20 B9
C2A0:3C BC 20 53 B8 AC 35 03 92
C2A8:AE 34 03 20 D4 BB EE 36 9E
C2B0:03 AD 34 03 18 69 05 8D D7
C2B8:34 03 AD 35 03 69 00 8D 6D
C2C0:35 03 AD AB 02 CD 36 03 C8
C2C8:B0 C5 EE 38 03 AD AA 02 9F
C2D0:CD 38 03 90 08 A9 00 8D 29
C2D8:36 03 4C 8F C2 A9 CA 85 94
C2E0:7A A9 CF 85 7B 20 8B B0 84
C2E8:8D B7 02 8C B8 02 A9 D0 1E
```

```
C2F0:85 7A A9 CF 85 7B 20 8B EF
C2F8:B0 8D B9 02 8C BA 02 A9 8E
C300:D6 85 7A A9 CF 85 7B 20 E9
C308:8B B0 8D C7 02 8C C8 02 85
C310:A9 DD 85 7A A9 CF 85 7B 4F
C318:20 8B B0 8D C3 02 8C C4 85
C320:02 A9 E5 85 7A A9 CF 85 C7
C328:7B 20 8B B0 8D C5 02 8C 06
C330:C6 02 A9 ED 85 7A A9 CF E8
C338:85 7B 20 8B B0 8D CB 02 73
C340:8C CC 02 AD B3 02 38 E5 58
C348:2F 8D B3 02 AD B4 02 E5 8B
C350:30 8D B4 02 AD B5 02 38 8A
C358:E5 2F 8D B5 02 AD B6 02 E1
C360:E5 30 8D B6 02 AD B7 02 3C
C368:38 E5 2F 8D B7 02 AD B8 1E
C370:02 E5 30 8D B8 02 AD B9 34
C378:02 38 E5 2F 8D B9 02 AD C3
C380:BA 02 E5 30 8D BA 02 AD AE
C388:BF 02 38 E5 2F 8D BF 02 07
C390:AD C0 02 E5 30 8D C0 02 BF
C398:AD C1 02 38 E5 2F 8D C1 F3
C3A0:02 AD C2 02 E5 30 8D C2 DA
C3A8:02 AD C3 02 38 E5 2F 8D 7A
C3B0:C3 02 AD C4 02 E5 30 8D 32
C3B8:C4 02 AD C5 02 38 E5 2F 21
C3C0:8D C5 02 AD C6 02 E5 30 D5
C3C8:8D C6 02 AD C7 02 38 E5 80
C3D0:2F 8D C7 02 AD C8 02 E5 E6
C3D8:30 8D C8 02 AD C9 02 38 E5
C3E0:E5 2F 8D C9 02 AD CA 02 D3
C3E8:E5 30 8D CA 02 AD CB 02 2E
C3F0:38 E5 2F 8D CB 02 AD CC 5B
C3F8:02 E5 30 8D CC 02 AD CD 51
C400:02 38 E5 2D 8D AD 02 AD FC
C408:AE 02 E5 2E 8D AE 02 AD E1
C410:B1 02 38 E5 2D 8D B1 02 5D
C418:AD B2 02 E5 2E 8D B2 02 D2
C420:AD AF 02 38 E5 2D 8D AF DE
C428:02 AD B0 02 E5 2E 8D B0 08
C430:02 AD D2 02 38 E5 2D 8D E1
C438:D2 02 AD D3 02 E5 2E 8D 30
C440:D3 02 AD A7 02 85 7A AD 2D
C448:A8 02 85 7B 60 A9 00 8D 46
C450:BB 02 20 1C CB F0 18 20 70
C458:FD AE 20 9E B7 8E BB 02 EB
C460:E0 00 F0 8D AD AC 02 CD EA
C468:BB 02 B0 03 4C 48 B2 A9 29
```

Another Example

The encoding problem is another classic example of the type of problem the back propagation network can solve. The problem is this: You need to associate four different input patterns with four different output patterns. For the BP network to correctly learn these patterns, it needs to be able to store a representation of these patterns in its middle layer (that is, it needs to learn the correct internal representation). The minimum number of PEs in the middle layer can have to represent the association of four different patterns is 2 (see Figure 1). ENCODE correctly learns to encode the four different patterns in its middle layer of PEs.

Although back propagation solves many of the problems of the linear associator, it does introduce many new ones. The major drawback of BP neural networks is the long training time. As you can see, the training times for a simple model, such as XOR, is one minute, 20 seconds. Unfortunately the times do not increase in a simple linear fashion, but rather in an exponential one, based on the number of PEs you have in each layer. The training time for the encoding example is 27 minutes, 49 seconds—nearly half an hour.

The most important problem to those in cognitive science is that the back propagation neural network is not based on neurophysiological data. It was developed to solve the problems of the linear associator. The next article in the series will examine a group of more biologically plausible models, competitive learning neural networks.

Future Computing

Program 1. BP (cont.)

```

C470:01 8D 36 03 A9 00 8D 38 75
C478:03 20 3C BC A2 CD A0 02 6E
C480:20 D4 BB AD BF 02 18 65 3D
C488:2F 8D 34 03 AD C0 02 65 9D
C490:30 8D 35 03 AE 38 03 AC 75
C498:36 03 AD AA 02 20 AA C6 0B
C4A0:AD 34 03 AC 35 03 20 A2 D1
C4A8:BB AD C9 02 18 65 2F 8D 17
C4B0:34 03 AD CA 02 65 30 8D 0B
C4B8:35 03 AE BB 02 AC 38 03 65
C4C0:AD A9 02 20 AA C6 AD 34 CD
C4C8:03 AC 35 03 20 28 BA A9 96
C4D0:CD A0 02 20 67 BB EE 38 DF
C4D8:03 AD A9 02 CD 38 03 B0 AA
C4E0:9B A9 F2 A0 CF 20 50 B8 63
C4E8:20 ED BF 20 0C BC A9 01 9F
C4F0:20 3C BC 20 6A B8 A9 F7 B4
C4F8:A0 CF 20 0F BB AD B3 02 B9
C500:18 65 2F 8D 34 03 AD B4 6D
C508:02 65 30 8D 35 03 AD 36 14
C510:03 20 90 C6 AE 34 03 AC 9C
C518:35 03 20 D4 BB EE 36 03 59
C520:AD AA 02 CD 36 03 90 03 2C
C528:4C 74 C4 A9 01 8D 36 03 D7
C530:A9 00 8D 38 03 20 3C BC 93
C538:A2 CD A0 02 20 D4 BB AD 36
C540:03 18 65 2F 8D 34 03 A1
C548:AD C2 02 65 30 8D 35 03 17
C550:AE 38 03 AC 36 03 AD AB 31
C558:02 20 AA C6 AD 34 03 AC 9F
C560:35 03 20 A2 BB AD B3 02 73
C568:18 65 2F 8D 34 03 AD B4 D5
C570:02 65 30 8D 35 03 AD 38 7E
C578:03 20 90 C6 AD 34 03 AC FC
C580:35 03 20 28 BA A9 CD A0 A6
C588:02 20 67 BB EE 38 03 AD A1
C590:AA 02 CD 38 03 B0 A1 A9 F6
C598:F2 A0 CF 20 50 B8 20 ED 55
C5A0:BF 20 0C BC A9 01 20 3C 2F
C5A8:BC 20 6A B8 A9 F7 A0 CF B1
C5B0:20 0F BB AD B5 02 18 65 AD
C5B8:2F 8D 34 03 AD B6 02 65 A7
C5C0:30 8D 35 03 AD 36 03 20 0B
C5C8:90 C6 AE 34 03 AC 35 03 9F
C5D0:20 D4 BB EE 36 03 AD AB CC
C5D8:02 CD 36 03 90 03 4C 30 29
C5E0:C5 A9 00 20 3C BC A9 01 E4
C5E8:8D 36 03 A2 CD A0 02 20 68
C5F0:D4 BB AD B5 02 18 65 2F 51
C5F8:8D 34 03 AD B6 02 65 30 4C
C600:8D 35 03 AD 36 03 20 90 6B
C608:C6 AD 34 03 AC 35 03 20 7B
C610:A2 BB AD C7 02 18 65 2F 7A
C618:8D 34 03 AD C8 02 65 30 FD
C620:8D 35 03 AD AB 02 AC 36 F1
C628:03 AE BB 02 20 AA C6 AD 61
C630:34 03 AC 35 03 20 50 B8 73
C638:A2 D8 A0 02 20 D4 BB A9 F6
C640:D8 A0 02 20 28 BA A9 CD F1
C648:A0 02 20 67 BB EE 36 03 12
C650:AD AB 02 CD 36 03 B0 93 6F
C658:AD CB 02 18 65 2F 8D 34 A8
C660:03 AD CC 02 65 30 8D 35 D0
C668:03 AD BB 02 20 90 C6 AE F9
C670:34 03 AC 35 03 20 D4 BB BF
C678:A9 02 20 3C BC AD 34 03 2B
C680:AC 35 03 20 0F BB AE 34 0D
C688:03 AC 35 03 20 D4 BB 60 C5
C690:AA E8 CA F0 14 AD 34 03 D8
C698:18 69 05 8D 34 03 AD 35 44
C6A0:03 69 00 8D 35 03 4C 92 C3
C6A8:C6 F0 8D FD 03 98 48 E8 37
C6B0:CA F0 2C AC FD 03 C8 AD 6B
C6B8:34 03 18 69 05 8D 34 03 84
C6C0:AD 35 03 69 00 8D 35 03 0D
C6C8:88 F0 E5 AD 34 03 18 69 B5
C6D0:05 8D 34 03 AD 35 03 69 AC
C6D8:00 8D 35 03 4C C8 C6 68 1C
C6E0:4C 90 C6 20 D4 BB A0 FF BA
C6E8:C8 B9 00 01 D0 FA C8 98 F5
C6F0:48 A9 00 85 22 A9 01 85 A4
C6F8:23 68 20 24 AB 60 20 57 EE
C700:C4 4C 07 C7 20 6F C4 A9 54
C708:01 8D 36 03 AD C7 02 18 1B

```

```

C710:65 2F 8D 34 03 AD C8 02 75
C718:65 30 8D 35 03 AD AB 02 93
C720:AC 36 03 AE BB 02 20 AA AF
C728:C6 AD 34 03 AC 35 03 20 9D
C730:8C BA AD B5 02 18 65 2F 30
C738:8D 34 03 AD B6 02 65 30 8F
C740:8D 35 03 AD 36 03 20 90 AD
C748:C6 AD 34 03 AC 35 03 20 BD
C750:A2 BB 20 53 B8 AD 34 03 41
C758:AC 35 03 20 28 BA A2 CD 2D
C760:A0 02 20 D4 BB AD 34 03 12
C768:AC 35 03 20 A2 BB A9 F7 4D
C770:A0 CF 20 50 B8 A9 CD A0 F5
C778:02 20 28 BA AD B9 02 18 32
C780:65 2F 8D 34 03 AD BA 02 C9
C788:65 30 8D 35 03 AD 36 03 1A
C790:20 90 C6 AE 34 03 AC 35 54
C798:03 20 D4 BB A9 00 8D 38 A8
C7A0:03 AD AD 02 18 65 2D 48 EB
C7A8:AD AE 02 65 2E A8 68 20 56
C7B0:A2 BB AD B9 02 18 65 2F 3C
C7B8:8D 34 03 AD BA 02 65 30 30
C7C0:8D 35 03 AD 36 03 20 90 2E
C7C8:C6 AD 34 03 AC 35 03 20 3E
C7D0:28 BA AD B3 02 18 65 2F 7E
C7D8:8D 34 03 AD B4 02 65 30 20
C7E0:8D 35 03 AD 38 03 20 90 5E
C7E8:C6 AD 34 03 AC 35 03 20 5E
C7F0:28 BA A2 CD A0 02 20 D4 96
C7F8:BB AD B1 02 18 65 2D 48 21
C800:AD B2 02 65 2E A8 68 20 B0
C808:A2 BB AD C5 02 18 65 2F 56
C810:8D 34 03 AD C6 02 65 30 E9
C818:8D 35 03 AD AB 02 AC 36 ED
C820:03 AE 38 03 20 AA C6 AD FC
C828:34 03 AC 35 03 20 28 BA 21
C830:A9 CD A0 02 20 67 B8 AE FC
C838:34 03 AC 35 03 20 D4 BB 8B
C840:AD C1 02 18 65 2F 8D 34 12
C848:03 AD C2 02 65 30 8D 35 7B
C850:03 AD AB 02 AC 36 03 AE 57
C858:38 03 20 AA C6 AD 34 03 CD
C860:AC 35 03 20 67 B8 AE 34 A7
C868:03 AC 35 03 20 D4 BB EE 38
C870:38 03 AD AA 02 CD 38 03 F9
C878:90 03 4C A1 C7 EE 36 03 D2
C880:AD AB 02 CD 36 03 90 03 D2
C888:4C 0C C7 A9 01 8D 36 03 84
C890:A9 00 A0 05 99 CC 02 88 E7
C898:D0 FA A9 01 8D 38 03 AD 97
C8A0:B9 02 18 65 2F 8D 34 03 04
C8A8:AD BA 02 65 30 8D 35 03 7B
C8B0:AD 38 03 20 90 C6 AD 34 B8
C8B8:03 AC 35 03 20 A2 BB AD 7E
C8C0:C1 02 18 65 2F 8D 34 03 28
C8C8:AD C2 02 65 30 8D 35 03 9D
C8D0:AD AB 02 AC 38 03 AE 36 90
C8D8:03 20 AA C6 AD 34 03 AC A6
C8E0:35 03 20 28 BA A9 CD A0 0D
C8E8:02 20 67 B8 A2 CD A0 02 8B
C8F0:20 D4 BB EE 38 03 AD AB 03
C8F8:02 CD 38 03 B0 A1 AD B3 51
C900:02 18 65 2F 8D 34 03 AD 2B
C908:B4 02 65 30 8D 35 03 AD 1B
C910:36 03 20 90 C6 AD 34 03 E4
C918:AC 35 03 20 A2 BB A9 F7 01
C920:A0 CF 20 50 B8 AD 34 03 E8
C928:AC 35 03 20 28 BA A9 CD 0F
C930:A0 02 20 28 BA AD B7 02 19
C938:18 65 2F 8D 34 03 AD B8 B1
C940:02 65 30 8D 35 03 AD 36 54
C948:03 20 90 C6 AE 34 03 AC DC
C950:35 03 20 D4 BB A9 00 8D A2
C958:38 03 AD AD 02 18 65 2D C1
C960:48 AD AE 02 65 2E A8 68 17
C968:20 A2 BB AD B7 02 18 65 62
C970:2F 8D 34 03 AD B8 02 65 6F
C978:30 8D 35 03 AD 36 03 20 CA
C980:90 C6 AD 34 03 AC 35 03 3F
C988:20 28 BA AD C9 02 18 65 54
C990:2F 8D 34 03 AD CA 02 65 D7
C998:30 8D 35 03 AD A9 02 AC 43
C9A0:38 03 AE BB 02 20 AA C6 4F
C9A8:AD 34 03 AC 35 03 20 28 69
C9B0:BA A2 CD A0 02 20 D4 BB 04

```

```

C9B8:AD B1 02 18 65 2D 48 AD 6F
C9C0:B2 02 65 2E A8 68 20 A2 87
C9C8:BB AD C3 02 18 65 2F 8D 80
C9D0:34 03 AD C4 02 65 30 8D D4
C9D8:35 03 AD AA 02 AC 36 03 5A
C9E0:AE 38 03 20 AA C6 AD 34 3C
C9E8:03 AC 35 03 20 28 BA A9 C0
C9F0:CD A0 02 20 67 B8 AE 34 85
C9F8:03 AC 35 03 20 D4 BB AD 89
CA00:BF 02 18 65 2F 8D 34 03 6A
CA08:AD C0 02 65 30 8D 35 03 60
CA10:AD AA 02 AC 36 03 AE 38 85
CA18:03 20 AA C6 AD 34 03 AC E9
CA20:35 03 20 67 B8 AE 34 03 77
CA28:AC 35 03 20 D4 BB EE 38 6F
CA30:03 AD A9 02 CD 38 03 90 ED
CA38:03 4C 5A C9 EE 36 03 AD 4E
CA40:AA 02 CD 36 03 90 03 4C 75
CA48:90 C8 60 A9 00 20 3C BC B4
CA50:AD D2 02 18 65 2D 8D D4 03
CA58:02 AD D3 02 65 2E 8D D5 C9
CA60:02 AE D4 02 AC D5 02 20 3E
CA68:D4 BB A9 01 8D BB 02 20 1C
CA70:04 C7 AD CB 02 18 65 2F D6
CA78:8D 34 03 AD CC 02 65 30 86
CA80:8D 35 03 AD BB 02 20 90 1C
CA88:C6 AD 34 03 AC 35 03 20 04
CA90:A2 BB AD D4 02 AC D5 02 D9
CA98:20 67 B8 AE D4 02 AC D5 F7
CAA0:02 20 D4 BB EE BB 02 AD AD
CAA8:AC 02 CD BB 02 B0 C0 60 3F
CAB0:20 FD AE 28 9E B7 8E FC 9B
CAB8:03 AD AF 02 18 65 2D 8D 8F
CAC0:D6 02 AD B0 02 65 2E 8D 92
CAC8:D7 02 20 4B CA 20 E1 FF 1E
CAD0:F0 2D AD FC 03 F0 11 AD 5B
CAD8:D4 02 AC D5 02 20 A2 BB DD
CAE0:20 E3 C6 A9 0D 20 D2 FF 81
CAE8:AD D4 02 AC D5 02 20 A2 2F
CAF0:BB AD D6 02 AC D7 02 20 B3
CAF8:5B BC C9 01 F0 CC 60 A0 D0
CB00:00 4C 38 A8 20 FD AE 20 B2
CB08:9E B7 8E BB 02 E0 00 F0 EE
CB10:08 AD AC 02 CD BB 02 B0 DE
CB18:03 4C 48 B2 20 FD AE 20 EE
CB20:9E AD 20 8F AD 20 A6 B6 61
CB28:CD A9 02 D0 EC 8E CD 02 9D
CB30:8C CE 02 A9 00 8D 36 03 42
CB38:A2 01 AC 36 03 AD CD 02 C6
CB40:85 22 AD CE 02 85 23 B1 E3
CB48:22 C9 31 F0 02 A2 00 8A BD
CB50:20 3C BC AD C9 02 18 65 65
CB58:2F 8D 34 03 AD CA 02 65 A3
CB60:30 8D 35 03 AD A9 02 AC 0F
CB68:36 03 C8 AE BB 02 20 AA B0
CB70:C6 AE 34 03 AC 35 03 20 2E
CB78:D4 BB EE 36 03 AD A9 02 CE
CB80:CD 36 03 D0 B3 AD BB 02 C7
CB88:F0 69 20 FD AE 20 9E AD B7
CB90:20 8F AD 20 A6 B6 CD AB 2B
CB98:02 D0 59 8E CD 02 8C CE D7
CBA0:02 A9 00 8D 36 03 A2 01 80
CBA8:AC 36 03 AD CD 02 85 22 03
CBB0:AD CE 02 85 23 B1 22 C9 59
CBB8:31 F0 02 A2 00 8A 20 3C 36
CBC0:BC AD C7 02 18 65 2F 8D 7D
CBC8:34 03 AD C8 02 65 30 8D 11
CBD0:35 03 AD AB 02 AC 36 03 66
CBD8:C8 AE BB 02 20 AA C6 AE FF
CBE0:34 03 AC 35 03 20 D4 BB 3A
CBE8:EE 36 03 AD AB 02 CD 36 F7
CBF0:03 D0 B3 60 4C 48 B2 20 C3
CBF8:0E CF 20 FD AE 20 9E AD 50
CC00:20 8F AD 20 A6 B6 8D 36 A6
CC08:03 A0 00 B1 22 99 D0 02 9B
CC10:C8 CC 36 03 F0 04 CD 14 65
CC18:D0 F1 A9 2C 99 D0 02 C8 9F
CC20:A9 57 99 DD 02 C8 98 A2 7C
CC28:DD A0 02 20 BD FF A9 01 5D
CC30:A2 08 A0 02 20 BA FF 20 5D
CC38:C0 FF A2 0F 20 C6 FF 20 B3
CC40:CF FF C9 30 F0 03 4C AD 77
CC48:CD 20 CC FF A2 01 20 C9 8D
CC50:FF AD A9 02 20 D2 FF AD A4
CC58:AA 02 20 D2 FF AD AB 02 09

```

Program 1. BP (cont.)

```

CC60:20 D2 FF AD AC 02 20 D2 1A
CC68:FF AE AD 02 AC AE 02 20 C7
CC70:F9 CC AE B1 02 AC B2 02 55
CC78:20 F9 CC AE AF 02 AC B0 B4
CC80:02 20 F9 CC AD BF 02 85 25
CC88:22 AD C0 02 85 23 AC AA 93
CC90:02 AE A9 02 20 11 CD AD BA
CC98:C1 02 85 22 AD C2 02 85 68
CCA0:23 AC AB 02 AE AA 02 20 D0
CCA8:11 CD AD C3 02 85 22 AD 48
CCB0:C4 02 85 23 AC AA 02 AE D2
CCB8:A9 02 20 11 CD AD C5 02 6F
CCC0:85 22 AD C6 02 85 23 AC E0
CCC8:AB 02 AE AA 02 20 11 CD B9
CCD0:AD C9 02 85 22 AD CA 02 AB
CCD8:85 23 AC A9 02 AE AC 02 54
CCE0:20 11 CD AD C7 02 85 22 D6
CCE8:AD C8 02 85 23 AC AB 02 49
CCF0:AE AC 02 20 11 CD 4C 20 C7
CCF8:CF 8A 18 65 2D 85 22 98 D2
CD00:65 2E 85 23 AC 00 B1 22 47
CD08:20 D2 FF C8 C0 05 D0 F6 A7
CD10:60 A5 2F 18 65 22 85 22 8D
CD18:A5 30 65 23 85 23 C8 8C 48
CD20:36 03 8C CD 02 E8 8E 38 0F
CD28:03 A0 00 A2 05 B1 22 20 EA
CD30:D2 FF C8 D0 02 E6 23 CA 18
CD38:D0 F3 A2 05 CE 36 03 D0 04
CD40:EC AD CD 02 8D 36 03 CE B1
CD48:38 03 D0 E1 60 20 D2 FF 22
CD50:20 CF FF C9 0D D0 F6 20 46
CD58:D2 FF 20 E7 FF 20 7A A6 FB
CD60:4C 74 A4 20 0E CF 20 FD C3
CD68:AE 20 9E AD 20 8F AD 20 CC
CD70:A6 B6 8D 36 03 A0 00 B1 6E
CD78:22 99 DD 02 C8 CC 36 03 50
CD80:F0 04 C0 14 D0 F1 A9 2C BC
CD88:99 DD 02 C8 A9 52 99 DD DC
CD90:02 C8 98 A2 DD A0 02 20 32
CD98:BD FF A9 01 A2 08 A0 02 D0
CDA0:20 BA FF 20 C0 FF A2 0F 57
CDA8:20 C6 FF 20 CF FF C9 30 4A
CDB0:D0 9B 20 CC FF A2 01 20 19
CDB8:C6 FF 20 CF FF 8D A9 02 44
CDC0:20 CF FF 8D AA 02 20 CF A6
CDC8:FF 8D AB 02 20 CF FF 8D 2B
CDD0:AC 02 20 B9 CE AE AD 02 71
CDD8:AC AE 02 20 65 CE AE B1 2E
CDE0:02 AC B2 02 20 65 CE AE 02
CDE8:AF 02 AC B0 02 20 65 CE A7
CDF0:AD BF 02 85 22 AD C0 02 37
CDF8:85 23 AC AA 02 AE A9 02 80
CE00:20 7D CE AD C1 02 85 22 05
CE08:AD C2 02 85 23 AC AB 02 EA
CE10:AE AA 02 20 7D CE AD C3 38
CE18:02 85 22 AD C4 02 85 23 93
CE20:AC AA 02 AE A9 02 20 7D FC
CE28:CE AD C5 02 85 22 AD C6 48
CE30:02 85 23 AC AB 02 AE AA CC
CE38:02 20 7D CE AD C9 02 85 99
CE40:22 AD CA 02 85 23 AC A9 8F
CE48:02 AE AC 02 20 7D CE AD 8A
CE50:C7 02 85 22 AD C8 02 85 3F
CE58:23 AC AB 02 AE AC 02 20 94
CE60:7D CE 4C 20 CF 8A 18 65 3A
CE68:2D 85 22 98 65 2E 85 23 DD
CE70:A0 00 20 CF FF 91 22 C8 B2
CE78:C0 05 D0 F6 60 A5 2F 18 51
CE80:65 22 85 22 A5 30 65 23 08
CE88:85 23 C8 8C 36 03 8C CD 38
CE90:02 E8 8E 38 03 A0 00 A2 FB
CE98:05 20 CF FF 91 22 C8 D0 32
CEA0:02 E6 23 CA D0 F3 A2 05 AA
CEA8:CE 36 03 D0 EC AD CD 02 64
CEB0:8D 36 03 CE 38 03 D0 E1 41
CEB8:60 A5 37 A4 38 85 33 84 E3
CEC0:34 A5 2D A4 2E 85 2F 84 3C
CEC8:30 85 31 84 32 A5 7A 8D F8
CED0:A7 02 A5 7B 8D A8 02 A9 EB
CED8:8B 85 7A A9 CF 85 7B 20 33
CEE0:8B B0 8D AD 02 8C AE 02 9E
CEE8:A9 8E 85 7A A9 CF 85 7B 6A
CEF0:20 8B B0 8D B1 02 8C B2 D1
CEF8:02 A9 91 85 7A A9 CF 85 2C

```

```

CF00:7B 20 8B B0 8D AF 02 8C 9D
CF08:B0 02 20 E2 C0 60 A9 00 8D
CF10:20 BD FF A9 0F A2 08 A0 7D
CF18:0F 20 BA FF 20 C0 FF 60 03
CF20:20 CC FF A9 01 20 C3 FF AD
CF28:A9 0F 20 C3 FF 60 4F 32 F2
CF30:28 50 32 29 2C 4F 33 28 FD
CF38:50 33 29 2C 45 32 28 50 48
CF40:32 29 2C 45 33 28 50 33 2B
CF48:29 2C 57 31 28 50 32 2C 98
CF50:50 31 29 2C 57 32 28 50 70
CF58:33 2C 50 32 29 2C 4D 31 8F
CF60:28 50 32 2C 50 31 29 2C F6
CF68:4D 32 28 50 33 2C 50 32 62
CF70:29 2C 54 28 50 33 2C 4E B2
CF78:50 29 2C 49 4E 28 50 31 89

```

```

CF80:2C 4E 50 29 2C 45 28 4E 7B
CF88:50 29 00 52 41 00 4D 4F B3
CF90:00 45 50 00 50 31 00 50 23
CF98:32 00 50 33 00 4E 50 00 68
CFA0:4F 31 28 30 29 00 4F 32 56
CFA8:28 30 29 00 4F 33 28 30 55
CFB0:29 00 49 4E 28 30 2C 30 7D
CFB8:29 00 57 31 28 30 2C 30 75
CFC0:29 00 57 32 28 30 2C 30 8D
CFC8:29 00 45 32 28 30 29 00 1D
CFD0:45 33 28 30 29 00 54 28 02
CFD8:30 2C 30 29 00 4D 31 28 F3
CFE0:30 2C 30 29 00 4D 32 28 FD
CFE8:30 2C 30 29 00 45 28 30 D9
CFF0:29 00 00 00 00 00 00 81 A6
CFF8:00 00 00 00 54 45 00 00 50

```

Program 2. XOR

```

HR 10 REM COPYRIGHT 1990 COMPU
TE! PUBLICATIONS INC. -
{SPACE}ALL RIGHTS RESERV
ED
GP 20 PRINT "{CLR}{5}{N}":POKE5
3280,0:POKE53281,11
QQ 30 IF PEEK(49153)<>24 OR PE
EK(49157)<>196 THEN LOAD
"BP.ML",8,1
PC 40 X=RND(-33333)
SD 50 SYS 49152,2,2,1,4,0.25,0
.9,0.02
CJ 60 SYS 49167,1,"00","0"
QQ 70 SYS 49167,2,"10","1"
ER 80 SYS 49167,3,"01","1"
FC 90 SYS 49167,4,"11","0"
DQ 100 PRINT"LEARNING PATTERNS"
"
KS 110 PRINT
KD 120 PRINT"THE TOTAL ERROR I
S:"

```

```

BX 130 TI$="0000000"
GK 140 SYS 49164,1
CS 150 PRINT"TIME SPENT LEARNI
NG : ";TI$
KG 160 PRINT"{DOWN}RESULTS:
{DOWN}"
XG 170 SYS49155,"00"
HE 180 PRINT "0 XOR 0 =";INT(O
3(1)+0.5);
KH 190 PRINT " (";O3(1);")"
JJ 200 SYS49155,"10"
MF 210 PRINT "1 XOR 0 =";INT(O
3(1)+0.5);
JG 220 PRINT " (";O3(1);")"
DM 230 SYS49155,"01"
BJ 240 PRINT "0 XOR 1 =";INT(O
3(1)+0.5);
QE 250 PRINT " (";O3(1);")"
BX 260 SYS49155,"11"
DP 270 PRINT "1 XOR 1 =";INT(O
3(1)+0.5);
AK 280 PRINT " (";O3(1);")"

```

Program 3. ENCODE

```

HR 10 REM COPYRIGHT 1990 COMPU
TE! PUBLICATIONS INC. -
{SPACE}ALL RIGHTS RESERV
ED
GP 20 PRINT "{CLR}{5}{N}":POKE5
3280,0:POKE53281,11
QQ 30 IF PEEK(49153)<>24 OR PE
EK(49157)<>196 THEN LOAD
"BP.ML",8,1
CM 40 X=RND(-11111)
GQ 50 SYS 49152,4,2,4,4,0.25,0
.9,0.02
DX 60 SYS 49167,1,"1000","0010"
"
MR 70 SYS 49167,2,"0100","0001"
"
FQ 80 SYS 49167,3,"0010","1000"
"
DF 90 SYS 49167,4,"0001","0100"
"
DQ 100 PRINT"LEARNING PATTERNS"
"
KS 110 PRINT
KD 120 PRINT"THE TOTAL ERROR I
S:"
BX 130 TI$="0000000"
GK 140 SYS 49164,1
JJ 150 PRINT"TIME SPENT LEARNI
NG : ";TI$:PRINT"PRINT"
RESULTS:"
RF 160 PRINT"{3 SPACES}LAYER
{6 SPACES}LAYER
{7 SPACES}LAYER"
SK 170 PRINT"{4 SPACES}ONE
{8 SPACES}TWO{8 SPACES}
THREE"
AC 180 SYS49155,"1000"
CC 190 PRINT "1{2 SPACES}0

```

```

{2 SPACES}0{2 SPACES}0
{SPACE}-->";INT(O2(1)+0
.5);INT(O2(2)+0.5);"-->
";
DC 200 FOR I= 1 TO 4
HB 210 PRINT INT(O3(I)+0.5);
QD 220 NEXT I
PG 230 PRINT
MB 240 SYS49155,"0100"
MS 250 PRINT "0{2 SPACES}1
{2 SPACES}0{2 SPACES}0
{SPACE}-->";INT(O2(1)+0
.5);INT(O2(2)+0.5);"-->
";
PG 260 FOR I= 1 TO 4
XE 270 PRINT INT(O3(I)+0.5);
EH 280 NEXT I
DM 290 PRINT
QE 300 SYS49155,"0010"
CP 310 PRINT "0{2 SPACES}0
{2 SPACES}1{2 SPACES}0
{SPACE}-->";INT(O2(1)+0
.5);INT(O2(2)+0.5);"-->
";
GM 320 FOR I= 1 TO 4
JK 330 PRINT INT(O3(I)+0.5);
XP 340 NEXT I
GR 350 PRINT
FK 360 SYS49155,"0001"
JG 370 PRINT "0{2 SPACES}0
{2 SPACES}0{2 SPACES}1
{SPACE}-->";INT(O2(1)+0
.5);INT(O2(2)+0.5);"-->
";
SS 380 FOR I= 1 TO 4
AR 390 PRINT INT(O3(I)+0.5);
CS 400 NEXT I
SB 410 PRINT

```

Future Computing Neural Networks

Part 3

Kevin E. Martin

In the final installment, the author presents the competitive learning model and a program to go with it.

Competitive learning (CL) is the focus of most of today's neural-network research. After you read about how the algorithm works, you can experiment with the program at the end of the article. The program implements the competitive-learning algorithm with a complete interface to BASIC.

In February, I discussed the back-propagation algorithm. It is in the associator class of neural-network models, but it was not developed from evidence from neurophysiology or even from psychology. The CL model is a much more biologically plausible model, and it exhibits many of the features we find in our own brain.

Competitive learning belongs to the class of self-organizing neural networks. These models take the salient features of the input domain and organize themselves so as to internally represent these features. Because they detect the features of the input domain, they can learn how to statistically classify the domain into several categories. This is especially useful when the network has to learn how to recognize damaged or distorted objects or patterns.

Like back propagation, competitive learning has layers of PEs (Processing Elements) and connections between the layers, but the similarities stop there. Each of the layers in the CL networks are broken up into groups of PEs where no PE can belong to more than one group. All of the PEs in these groups receive their input from the previous layer, and then they compete to see who gets to output from their group. Only one PE can output its signal from its group.

There is no teacher vector in CL, because the network organizes itself into recognizing the features of the input domain. This is quite a bit more like our own brain, since the vast majority of the cells in our own brain have no connection directly to the outside world and therefore do

not receive any teacher input. We can adapt to almost any type of environment, and the CL neural network is leading us toward developing a neural-network model that actually can adapt itself to any environment.

Another feature different from the previous networks we have seen is that we have connections not only between layers, but also within a layer. These intralayer connections are only between PEs within the same group, and they are inhibitory. So, if one PE gets a stronger input signal from the previous layer, it will inhibit its neighbors. In doing so, it becomes stronger and is said to be "winning the competition." Then, after a winner is determined, it outputs its signal to the next layer. The connections between layers are all excitatory, which will cause another competition to occur in the next layer.

The learning algorithm is very simple. All the weights in the connection matrix which lead from all the cells in the previous layer to a single cell in the current layer (that is, a row in the connection matrix) must sum to 1. This keeps the weights evenly distributed, and no PE in the group will get too strong. When a PE in a group wins a competition, it gets to learn, and the weights are changed according to a mathematical learning rule. Since all the weights must sum to 1, when a PE learns, the weight for connection between the current PE and the active PEs in the previous layer becomes stronger, and the weight for connection between the current PE and the inactive PEs becomes weaker. This causes the PE that won the competition to become more likely to recognize the pattern when it appears again.

Although competitive-learning neural networks are more like the brain in many respects, they still have several limitations. First, each of the clusters should learn different patterns, but in this scheme, there are no provisions which keep them from learning the same patterns. If two

clusters' weights are similar, they will tend to learn the same pattern. A simple fix for this problem is to have each of the clusters repel each other (that is, to have inhibitory connections between the clusters).

The linear-associator and back-propagation neural networks were able to associate two different patterns. This is a very important type of learning. The competitive-learning neural network can't learn to associate two patterns, but if it were combined with another network scheme, such as back propagation, then we would have a very powerful network architecture able to classify patterns and associate these classifications as well as the patterns themselves.

Since the size of the competitive-learning networks are limited to a certain number of clusters, they can classify the inputs into a limited number of categories. This limitation does not become apparent until the number of natural classifications of the inputs exceeds the number of clusters the CL network contains. In this situation, the network will classify the extra classes into categories in which they do not belong. This can be solved by simply adding more PEs and more clusters in the layers.

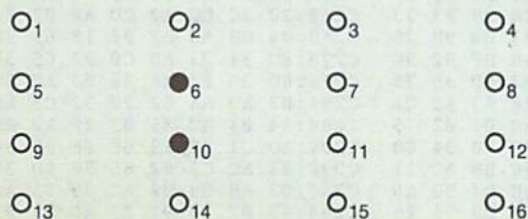
The largest problem, and the most difficult to solve, is the problem of stability. The CL network should, after a certain amount of time, stabilize each of its weights to a final value. After this, the weight change from subsequent training should not move the weights. But the weights in a CL network significantly change even after a long period of training. This problem has been solved by another neural network, Adaptive Resonance Theory (ART). For a discussion of ART, see *Neural Computing: Theory and Practice*, by Philip Wasserman.

The field of neural computing has grown rapidly since its rebirth in 1982, but there is still much more research to be done. Over the next several years, many new neural-network architectures will be developed and implemented. The applications will become wider and wider, but we are still a very long way from the day when every home has a computer to talk to.

Hands On

The accompanying program performs graph partitioning. It takes inputs and classifies them into two distinct groups (one for each PE in the second layer). The first layer of PEs is arranged in a grid shape shown in Figure 1. The inputs are dipoles; that is, only two PEs in the input pattern are allowed to be 1, and the rest are set to 0. In order to determine the correct classifications, there must be some order to the input patterns. To accomplish this, we limit the inputs to dipoles which are adjacent to each other along a horizontal or vertical line of the grid. As shown in Figure 1, only two PEs are active (blackened) and the rest are inactive (open). There are 24 possible dipole inputs, and each of these must be presented during a training run.

Figure 1: Graph Partitioning



What's startling about this example is that the program classifies the inputs into two distinct groups without any teacher. The groups are equally distributed; that is, they accept approximately equal numbers of patterns. They are also divided into very logical halves. If we were asked to divide the grid shown in Figure 1 into two distinct, equally distributed groups, we would most likely divide them between the second and third columns or between the second and third rows. This is exactly what the graph-partitioning example does. It divides the graph (the grid) into two halves horizontally or vertically and sometimes (though not very often) along the diagonal.

The competitive-learning example program is divided into two parts. The majority of the work is performed by the machine language routines. To type these in, you'll need MLX, the machine language entry program found elsewhere in this issue. The MLX prompts, and the values you should enter, are as follows:

Starting address: C000
Ending address: C98F

Once you've finished entering the data, save a copy of the routines to disk with the name CL.ML. The BASIC program expects to find a file with this name.

The graph-partitioning program is written in BASIC; it calls the machine language routines. To avoid typing errors, you should use *The Automatic Proofreader*, also found in this issue, while entering it. Save a copy of the program to the same disk that contains CL.ML.

The CL example program, Dipole, is very simple. In line 40, I initialize the random-number generator. The CL network is initialized by calling the machine language routine at 49152 with the following parameters: the number of PEs in the first layer, the number of PEs in the second layer, the number of patterns to learn, and the learning rate. An example of this is shown in line 60 of the example program.

This initialization sets up all the variables used by the program. These variables are standard BASIC variables and arrays and are described in the following table.

Variable	Meaning
p1	Number of PEs in layer 1
p2	Number of PEs in layer 2
np	Number of patterns for the system to learn
rate	Learning rate
Array (Size)	Meaning
o2(p2)	Output from layer 2
w1(p2,p1)	Connection weights between layers 1 and 2
in(p1,np)	Input patterns

Remember that these variables and arrays are erased if you perform a NEW or CLEAR command, load another program, or change the existing program.

Before training the network, all of the input patterns must be presented to the system. To do this, use the command SYS 49167 followed by the number of the pattern and then the pattern itself. The pattern should consist of 1s and 0s and should be enclosed within quotation marks. The first number corresponds to the activation of the upper left PE in the input layer. The rest of the PEs are numbered from left to right as shown in Figure 1. (See lines 80-310 for an example of this.)

Now the CL network is ready to learn. To teach it the patterns, simply enter SYS 49164 followed by the number of times you want the patterns to be presented. If you

Future Computing

specify a large number, the program will take some time to complete because each time you present a set of patterns it takes nearly 15 seconds. In order to properly train the network, you need to train it several hundred times. For 400 training runs, it takes over 60 minutes to train the network.

Once the network has been trained, you should save the current values so that another long training run will be unnecessary. To do this, type SYS 49170 followed by the filename in quotation marks. To reload the network, use SYS 49173 followed by the filename in quotation marks.

Now that the network has been trained and saved for further study, the network will correctly classify patterns according to the rules it derived. It will correctly classify all the patterns you used to teach it, but it will also classify patterns you did not teach it. This is one of the most powerful features of the CL network. If you want the program to recognize a pattern, type SYS 49155 followed by the input pattern in quotation marks. The input pattern should consist of 1s and 0s, and its length should be equal to p1 (the number of PEs in the first layer). After this, the output pattern will be in o2.

In the graph-partitioning program, the display consists of four parts. The first part consists of a 4 × 4 grid of weights on the connections between the PEs in the first layer and the first PE in the second layer. The next part is the total weight of all the weights on the connections in the 4 × 4 grid above it. The third part consists of another 4 × 4 grid of weights on the connections between the PEs in the first layer and the second PE in the second layer. The final part is the total weight of all the weights on the connections in the second grid. Each of these numbers is out of 1,000,000.

The total of all the weights should remain at or close to 1,000,000. This keeps the symmetry of the distribution of the weights and is vital to the functioning of the CL network. As the program progresses, the weights will become stronger on one side of the grid. Each grid should have weights that look like mirror images of each other. Figure 2 shows the weights after 400 training trials. Notice that the grid has been partitioned into two groups. If a di-

Figure 2: Weights

63653	67248	0	0
122666	141174	3283	0
142081	192964	61438	0
87442	91651	26394	0
999994			
0	40386	135818	67980
0	49159	200137	125781
0	2167	91572	108784
0	0	80543	97667
999994			

pole that is on the left side of the grid is presented, the first PE in the second layer will be active. If it is on the right side of the grid, the second PE in the second layer will be active.

Sources

If you're intrigued by neural computing, check your library or university bookstore for the books listed below.

Neural Computing: Theory and Practice

By Philip Wasserman

From Van Nostrand Reinhold

Neurocomputing: Foundations of Research

Edited by James A. Anderson and Edward Rosenfeld

From MIT Press

Parallel Distributed Processing (two volumes)

Explorations in Parallel Distributed Processing

By Rumelhart, McClelland, and the PDP Research Group

From MIT Press

Note: To understand the models presented in these books, you'll need to have an understanding of linear algebra and elementary calculus.

CL

C000:4C 18 C0 4C 47 C2 EA EA 90
 C008:60 EA EA 60 4C FD C5 4C 0A
 C010:31 C6 4C E8 C6 4C F0 C7 05
 C018:20 FD AE 20 9E B7 8E A9 9B
 C020:02 20 FD AE 20 9E B7 8E CE
 C028:AA 02 20 FD AE 20 9E B7 4E
 C030:8E AC 02 20 FD AE A5 7A D6
 C038:8D A7 02 A5 7B 8D A8 02 6A
 C040:A9 58 85 7A A9 C9 85 7B FF
 C048:20 8B B0 8D AD 02 8C AE E8
 C050:02 AD A7 02 85 7A AD A8 6D
 C058:02 85 7B 20 8A AD AE AD C3
 C060:02 AC AE 02 20 D4 BB A5 75
 C068:7A 8D A7 02 A5 7B 8D A8 7E
 C070:02 A9 5B 85 7A A9 C9 85 B4
 C078:7B 20 8B B0 48 98 48 AD 1F
 C080:A9 02 20 3C BC 68 A8 68 60
 C088:AA 20 D4 BB A9 5E 85 7A 0A
 C090:A9 C9 85 7B 20 8B B0 48 9A
 C098:98 48 AD AA 02 20 3C BC 9E
 C0A0:68 A8 68 AA 20 D4 BB A9 AD
 C0A8:61 85 7A A9 C9 85 7B 20 A1
 C0B0:8B B0 48 98 48 AD AC 02 0B
 C0B8:20 3C BC 68 A8 68 AA 20 D3
 C0C0:D4 BB A9 35 85 7A A9 C9 57
 C0C8:85 7B AD 35 C9 20 81 B0 77
 C0D0:A9 64 85 7A A9 C9 85 7B 93
 C0D8:20 8B B0 8D B3 02 8C B4 AF

C0E0:02 A9 01 20 3C BC AE B3 D5
 C0E8:02 AC B4 02 20 D4 BB A9 C2
 C0F0:6A 85 7A A9 C9 85 7B 20 6E
 C0F8:8B B0 8D C1 02 8C C2 02 04
 C100:A9 01 8D 36 03 A9 00 8D F9
 C108:38 03 A9 72 85 7A A9 C9 F7
 C110:85 7B 20 8B B0 8D BF 02 2F
 C118:8C C0 02 AD 38 03 D0 11 AD
 C120:A9 00 20 3C BC AC C0 02 5C
 C128:AE BF 02 20 D4 BB 4C 75 D8
 C130:C1 A9 01 20 3C BC 20 97 CD
 C138:E0 AD BF 02 8D 34 03 AD A0
 C140:C0 02 8D 35 03 AC 36 03 E3
 C148:AE 38 03 AD AA 02 20 92 9C
 C150:C3 AC 35 03 AE 34 03 20 24
 C158:D4 BB AD BF 02 AC C0 02 2D
 C160:20 A2 BB AD 34 03 AC 35 2B
 C168:03 20 67 B8 AE BF 02 AC 13
 C170:C0 02 20 D4 BB EE 38 03 33
 C178:AD A9 02 CD 38 03 B0 9B 25
 C180:A9 01 8D 38 03 AD BF 02 9E
 C188:8D 34 03 AD C0 02 8D 35 79
 C190:03 AC 36 03 AE 38 03 AD C1
 C198:AA 02 20 92 C3 AD BF 02 75
 C1A0:AC C0 02 20 A2 BB AD 34 80
 C1A8:03 AC 35 03 20 0F BB AC 11
 C1B0:35 03 AE 34 03 20 D4 BB A6
 C1B8:EE 38 03 AD A9 02 CD 38 26
 C1C0:03 B0 C2 EE 36 03 AD AA FC
 C1C8:02 CD 36 03 90 08 A9 00 AF

C1D0:8D 38 03 4C 1B C1 A9 7A FB
 C1D8:85 7A A9 C9 85 7B 20 8B 75
 C1E0:B0 8D C3 02 8C C4 02 AD E1
 C1E8:B3 02 38 E5 2F 8D B3 02 45
 C1F0:AD B4 02 E5 30 8D B4 02 3A
 C1F8:AD BF 02 38 E5 2F 8D BF CD
 C200:02 AD C0 02 E5 30 8D C0 E5
 C208:02 AD C1 02 38 E5 2F 8D 97
 C210:C1 02 AD C2 02 E5 30 8D 6E
 C218:C2 02 AD C3 02 38 E5 2F 5D
 C220:8D C3 02 AD C4 02 E5 30 A2
 C228:8D C4 02 AD AD 02 38 E5 8C
 C230:2D 8D AD 02 AD AE 02 E5 97
 C238:2E 8D AE 02 AD A7 02 85 C3
 C240:7A AD A8 02 85 7B 60 A9 28
 C248:00 8D BB 02 20 49 C6 A9 26
 C250:01 8D 36 03 A9 00 8D C5 DE
 C258:02 A9 01 8D 38 03 A9 00 63
 C260:20 3C BC A2 CD A0 02 20 DB
 C268:D4 BB AD BF 02 18 65 2F 63
 C270:8D 34 03 AD C0 02 65 30 0E
 C278:8D 35 03 AE 38 03 AC 36 BA
 C280:03 AD AA 02 20 92 C3 AD E8
 C288:34 03 AC 35 03 20 A2 BB 6B
 C290:AD C1 02 18 65 2F 8D 34 56
 C298:03 AD C2 02 65 30 8D 35 BF
 C2A0:03 AE BB 02 AC 38 03 AD E4
 C2A8:A9 02 20 92 C3 AD 34 03 F0
 C2B0:AC 35 03 20 28 BA A9 CD 89
 C2B8:A0 02 20 67 B8 EE 38 03 7E

```

C2C0:AD A9 02 CD 38 03 B0 9B 6F
C2C8:AD C5 02 D0 08 A9 01 8D 5A
C2D0:C5 02 4C 24 C3 A2 CD A0 6A
C2D8:02 20 D4 BB AD B3 02 18 16
C2E0:65 2F 8D 34 03 AD B4 02 14
C2E8:65 30 8D 35 03 AD C5 02 8E
C2F0:20 78 C3 AD 34 03 AC 35 34
C2F8:03 20 A2 BB A9 CD A0 02 DF
C300:20 5B BC AB A9 00 20 3C 54
C308:BC 68 C9 01 F0 16 AE 34 C2
C310:03 AC 35 03 20 D4 BB AD 94
C318:36 03 8D C5 02 A9 CD A0 7C
C320:02 20 A2 BB AD B3 02 18 19
C328:65 2F 8D 34 03 AD B4 02 5D
C330:65 30 8D 35 03 AD 36 03 B9
C338:20 78 C3 AE 34 03 AC 35 8D
C340:03 20 D4 BB EE 36 03 AD AB
C348:AA 02 CD 36 03 90 03 4C 6F
C350:59 C2 AD B3 02 18 65 2F 90
C358:8D 34 03 AD B4 02 65 30 97
C360:8D 35 03 AD C5 02 20 78 26
C368:C3 A9 01 20 C3 BC AE 34 C4
C370:03 AC 35 03 20 D4 BB 60 A7
C378:AA E8 CA F0 14 AD 34 03 BA
C380:18 69 05 8D 34 03 AD 35 26
C388:03 69 00 8D 35 03 4C 7A 8D
C390:C3 60 8D FD 03 98 48 E8 97
C398:CA F0 2C AC FD 03 C8 AD 4D
C3A0:34 03 18 69 05 8D 34 03 66
C3A8:AD 35 03 69 00 8D 35 03 EE
C3B0:88 F0 E5 AD 34 03 18 69 97
C3B8:05 8D 34 03 AD 35 03 69 8E
C3C0:00 8D 35 03 4C B0 C3 68 97
C3C8:4C 78 C3 20 D0 BD A0 FF 36
C3D0:C8 B9 00 01 D0 FA C8 98 D7
C3D8:48 A9 00 85 22 A9 01 85 86
C3E0:23 68 20 24 AB 60 A9 01 8D
C3E8:8D 36 03 AD C3 02 18 65 BB
C3F0:2F 8D 34 03 AD C4 02 65 14
C3F8:30 8D 35 03 AD 36 03 20 3F
C400:78 C3 AD 36 03 20 3C BC 9D
C408:AE 34 03 AC 35 03 20 D4 EB
C410:BB EE 36 03 AD AC 02 CD 1C
C418:36 03 B0 CF A9 01 8D 36 33
C420:03 AC AC 02 C8 98 38 ED 13
C428:36 03 20 C3 BC A2 CD A0 02
C430:02 20 D4 BB A9 01 20 3C E6
C438:BC 20 97 E0 A9 CD A0 02 F0
C440:20 28 BA 20 CC BC A2 CD A9
C448:A0 02 20 D4 BB AD 36 03 F7
C450:20 3C BC A9 CD A0 02 20 40
C458:67 B8 20 A1 B7 8E 38 03 4D
C460:AD C3 02 18 65 2F 8D 34 AA
C468:03 AD C4 02 65 30 8D 35 D3
C470:03 AD 36 03 20 78 C3 AD F5
C478:34 03 8D CD 02 AC 35 03 9B
C480:8C CE 02 20 8C BA AD C3 B4
C488:02 18 65 2F 8D 34 03 AD A9
C490:C4 02 65 30 8D 35 03 AD A1
C498:38 03 20 78 C3 AD 34 03 CA
C4A0:AC 35 03 20 A2 BB AE CD 5F
C4A8:02 AC CE 02 20 D4 BB 20 44
C4B0:FC BB AE 34 03 AC 35 03 F8
C4B8:20 D4 BB EE 36 03 AD AC B3
C4C0:02 CD 36 03 90 03 4C 21 FF
C4C8:C4 A9 01 8D C6 02 AD C3 75
C4D0:02 18 65 2F 8D 34 03 AD F1
C4D8:C4 02 65 30 8D 35 03 AD E9
C4E0:C6 02 20 78 C3 AD 34 03 1A
C4E8:AC 35 03 20 A2 BB 20 A1 5E
C4F0:B7 8A 8D BB 02 20 4F C2 58
C4F8:AD C1 02 18 65 2F 8D 34 C2
C500:03 AD C2 02 65 30 8D 35 2D
C508:03 A0 00 AE BB 02 AD A9 13
C510:02 20 92 C3 AD 34 03 AC 24
C518:35 03 20 A2 BB 20 A1 B7 86
C520:E0 00 D0 03 4C EE C5 A9 B9
C528:01 8D 36 03 AD C1 02 18 1F
C530:65 2F 8D 34 03 AD C2 02 85
C538:65 30 8D 35 03 AD A9 02 AB
C540:A0 00 AE BB 02 20 92 C3 27
C548:AD 34 03 AC 35 03 20 A2 7B
C550:BB AD C1 02 18 65 2F 8D BF
C558:34 03 AD C2 02 65 30 8D 34
C560:35 03 AD A9 02 AC 36 03 C9
C568:AE BB 02 20 92 C3 AD 34 AF
C570:03 AC 35 03 20 0F BB AD E1
C578:AD 02 18 65 2D 8D 34 03 BF
C580:AD AE 02 65 2E 8D 35 03 3A
C588:AD 34 03 AC 35 03 20 28 41
C590:BA A2 CD A0 02 20 D4 BB DB
C598:AD 34 03 AC 35 03 20 A2 CB
C5A0:BB AD BF 02 18 65 2F 8D CF
C5A8:34 03 AD C0 02 65 30 8D 64
C5B0:35 03 AD AA 02 AC C5 02 48
C5B8:AE 36 03 20 92 C3 AD 34 BE
C5C0:03 AC 35 03 20 28 BA A9 90
C5C8:CD A0 02 20 50 B8 AD 34 9A
C5D0:03 AC 35 03 20 67 B8 AE 9E
C5D8:34 03 AC 35 03 20 D4 BB 26
C5E0:EE 36 03 AD A9 02 CD 36 D3
C5E8:03 90 03 4C 2C C5 EE C6 5C
C5F0:02 AD AC 02 CD C6 02 90 BC
C5F8:03 4C CE C4 60 20 FD AE 6D
C600:20 8A AD 20 AA B1 8D C8 F7
C608:02 8C C7 02 C0 00 D0 04 7E
C610:C9 00 F0 17 20 E6 C3 20 56
C618:E1 FF F0 10 CE C7 02 D0 20
C620:F3 AD C8 02 F0 05 CE C8 4E
C628:02 D0 E9 60 A0 00 4C 38 04
C630:A8 20 FD AE 20 9E B7 8E 3E
C638:BB 02 E0 00 F0 08 AD AC EF
C640:02 CD BB 02 B0 03 4C 48 4C
C648:B2 20 FD AE 20 9E AD 20 D8
C650:8F AD 20 A6 B6 CD A9 02 C1
C658:D0 EC 8E CD 02 8C CE 02 1A
C660:A9 00 8D 36 03 AD 38 03 99
C668:03 60 03 AD CD 02 85 22 B8
C670:AD CE 02 85 23 B1 22 C9 0F
C678:31 F0 04 A9 00 F0 05 EE B2
C680:38 03 A9 01 20 3C BC AD 49
C688:C1 02 18 65 2F 8D 34 03 EB
C690:AD C2 02 65 30 8D 35 03 61
C698:AD A9 02 AC 36 03 C8 AE 70
C6A0:BB 02 20 92 C3 AE 34 03 FD
C6A8:AC 35 03 20 D4 BB EE 36 E5
C6B0:03 AD A9 02 CD 36 03 D0 9E
C6B8:AF AD 38 03 20 3C BC AD D9
C6C0:C1 02 18 65 2F 8D 34 03 24
C6C8:AD C2 02 65 30 8D 35 03 99
C6D0:AD A9 02 A0 00 AE BB 02 1E
C6D8:20 92 C3 AE 34 03 AC 35 BA
C6E0:03 20 D4 BB 60 4C 48 B2 C5
C6E8:20 15 C9 20 FD AE 20 9E 90
C6F0:AD 20 8F AD 20 A6 B6 8D C0
C6F8:36 03 A0 00 B1 22 9D 9D 9D
C700:02 C8 CC 36 03 F0 04 C0 64
C708:14 D0 F1 A9 2C 99 DD 02 34
C710:C8 A9 57 99 DD 02 C8 98 14
C718:A2 DD A0 02 20 BD FF A9 46
C720:01 A2 08 A0 02 20 BA FF E9
C728:20 C0 FF A2 0F 20 C6 FF A8
C730:20 CF FF C9 30 F0 03 4C F7
C738:DA C7 20 CC FF A2 01 20 A4
C740:C9 FF AD A9 02 20 D2 FF 3B
C748:AD AA 02 20 D2 FF AD AC 3A
C750:02 20 D2 FF AE AD 02 AC 20
C758:AE 02 20 86 C7 AD BF 02 A2
C760:85 22 AD C0 02 85 23 AC 16
C768:AA 02 AE A9 02 20 9E C7 D3
C770:AD C1 02 85 22 AD C2 02 2F
C778:85 23 AC A9 02 AE AC 02 E9
C780:20 9E C7 4C 27 C9 8A 18 13
C788:65 2D 85 22 98 65 2E 85 25
C790:23 A0 00 B1 22 20 D2 FF 2C
C798:C8 C0 05 D0 F6 60 A5 2F 1E
C7A0:18 65 22 85 22 A5 30 65 9F
C7A8:23 85 23 C8 8C 36 03 8C EB
C7B0:CD 02 E8 8E 38 03 A0 00 BC
C7B8:A2 05 B1 22 20 D2 FF C8 48
C7C0:D0 02 E6 23 CA D0 F3 A2 6C
C7C8:05 CE 36 03 D0 EC AD CD E8
C7D0:02 8D 36 03 CE 38 03 D0 E9
C7D8:E1 60 20 D2 FF 20 CF FF C2
C7E0:C9 0D D0 F6 20 D2 FF 20 8E
C7E8:E7 FF 20 7A A6 4C 74 A4 0C
C7F0:20 15 C9 20 FD AE 20 9E 9A
C7F8:AD 20 8F AD 20 A6 B6 8D CA
C800:36 03 A0 00 B1 22 99 DD A8
C808:02 C8 CC 36 03 F0 04 C0 6E
C810:14 D0 F1 A9 2C 99 DD 02 3E
C818:C8 A9 52 99 DD 02 C8 98 7D
C820:A2 DD A0 02 20 BD FF A9 50
C828:01 A2 08 A0 02 20 BA FF F3
C830:20 C0 FF A2 0F 20 C6 FF B2
C838:20 CF FF C9 30 D0 9B 20 86
C840:CC FF A2 01 20 C6 FF 20 D8
C848:CF FF 8D A9 02 20 CF FF 3E
C850:8D AA 02 20 CF FF 8D AC DB
C858:02 20 E2 C8 AE AD 02 AC B8
C860:AE 02 20 8E C8 AD BF 02 35
C868:85 22 AD C0 02 85 23 AC 20
C870:AA 02 AE A9 02 20 A6 C8 EE
C878:AD C1 02 85 22 AD C2 02 39
C880:85 23 AC A9 02 AE AC 02 F3
C888:20 A6 C8 4C 27 C9 8A 18 3F
C890:65 2D 85 22 98 65 2E 85 2F
C898:23 A0 00 20 CF FF 91 22 A9
C8A0:C8 C0 05 D0 F6 60 A5 2F 28
C8A8:18 65 22 85 22 A5 30 65 A9
C8B0:23 85 23 C8 8C 36 03 8C F5
C8B8:CD 02 E8 8E 38 03 A0 00 C6
C8C0:A2 05 20 CF FF 91 22 C8 39
C8C8:D0 02 E6 23 CA D0 F3 A2 72
C8D0:05 CE 36 03 D0 EC AD CD F6
C8D8:02 8D 36 03 CE 38 03 D0 F3
C8E0:E1 60 A5 37 A4 38 85 33 E7
C8E8:84 34 A5 2D A4 2E 85 2F 69
C8F0:84 30 85 31 84 32 A5 7A 47
C8F8:8D A7 02 A5 7B 8D A8 02 3B
C900:A9 58 85 7A A9 C9 85 7B D1
C908:20 8B B0 8D AD 02 8C AE BA
C910:02 20 71 C0 60 A9 00 20 B0
C918:BD FF A9 0F A2 08 A0 0F 36
C920:20 BA FF 20 C0 FF 60 20 5B
C928:CC FF A9 01 20 C3 FF A9 21
C930:0F 20 C3 FF 60 4F 32 28 98
C938:50 32 29 2C 57 31 28 50 88
C940:32 2C 50 31 29 2C 49 4E EF
C948:28 50 31 2C 4E 50 29 2C 1F
C950:50 41 54 28 4E 50 29 00 6F
C958:52 41 00 50 31 00 50 32 C6
C960:00 4E 50 00 4F 32 28 30 55
C968:29 00 49 4E 28 30 2C 30 29
C970:29 00 57 31 28 30 2C 30 21
C978:29 00 50 41 54 28 30 29 8B
C980:00 00 00 00 00 00 81 00 17
C988:00 00 00 54 45 00 00 00 8B

```

BEFORE TYPING...

Before typing in programs,
please refer to "How to Type In
COMPUTE!'s Gazette Programs,"
elsewhere in this issue.

Dipole

```

HR 10 REM COPYRIGHT 1990 COMPU
TE! PUBLICATIONS INC. -
{SPACE}ALL RIGHTS RESERV
ED
GP 20 PRINT"{CLR}{5}{N}":POKE5
3280,0:POKE53281,11
MM 30 IF PEEK(49153)<>24 OR PE
EK(49157)<>194 THEN LOAD
"CL.ML",8,1
PC 40 X=RND(-33333)
PA 50 PRINT"INITIALIZING"
RR 60 SYS 49152,16,2,24,0.1
RQ 70 PRINT"LOADING PATTERNS"
KB 80 SYS 49167,1,"11000000000
00000"
AA 90 SYS 49167,2,"01100000000
00000"
PP 100 SYS 49167,3,"0011000000
00000"
GK 110 SYS 49167,4,"0000110000
00000"
BH 120 SYS 49167,5,"0000011000
00000"

```

Future Computing (cont.)

```

XE 130 SYS 49167,6,"0000001100
000000"
QC 140 SYS 49167,7,"0000000011
000000"
HA 150 SYS 49167,8,"0000000001
100000"
MB 160 SYS 49167,9,"0000000000
110000"
CF 170 SYS 49167,10,"0000000000
0001100"
SD 180 SYS 49167,11,"0000000000
0000110"
MB 190 SYS 49167,12,"0000000000
0000011"
ED 200 SYS 49167,13,"100010000
0000000"
AB 210 SYS 49167,14,"010001000
0000000"
QX 220 SYS 49167,15,"001000100
0000000"
JR 230 SYS 49167,16,"000100010
0000000"
EM 240 SYS 49167,17,"000010001
0000000"
AJ 250 SYS 49167,18,"000001000
1000000"
RG 260 SYS 49167,19,"000000100
0100000"
KQ 270 SYS 49167,20,"000000010
0010000"
FR 280 SYS 49167,21,"000000001
0001000"
BP 290 SYS 49167,22,"000000000
1000100"
SJ 300 SYS 49167,23,"000000000
0100010"
PG 310 SYS 49167,24,"000000000
0010001"
PA 320 PRINT "{CLR}"
FR 330 FOR I=1 TO 400

KF 340 SYS 49164,1
MC 350 T=0
AE 360 PRINT "{HOME}"
JJ 370 FOR J= 1 TO 16
XB 380 A=INT(W1(1,J)*1000000)
DG 390 T=T+A
SR 400 IF A < 10 THEN PRINT " "
;
DG 410 IF A < 100 THEN PRINT "
{SPACE}";
HR 420 IF A < 1000 THEN PRINT "
";
MA 430 IF A < 10000 THEN PRINT
" ";
KE 440 IF A < 100000 THEN PRIN
T " ";
DF 450 PRINT A,
FD 460 NEXT J
FR 470 PRINT:PRINT T
MG 480 PRINT
HQ 490 T=0
XB 500 FOR J= 1 TO 16
CM 510 A=INT(W1(2,J)*1000000)
FR 520 T=T+A
GF 530 IF A < 10 THEN PRINT " "
;
EX 540 IF A < 100 THEN PRINT "
{SPACE}";
RG 550 IF A < 1000 THEN PRINT "
";
BJ 560 IF A < 10000 THEN PRINT
" ";
AX 570 IF A < 100000 THEN PRIN
T " ";
QR 580 PRINT A,
SQ 590 NEXT J
AE 600 PRINT:PRINT T
DP 610 NEXT I
XQ 620 END

```



Looking for a Widget for your Printer and need it now? Call Precision!

Precision Images stocks a complete selection of parts, supplies, and manuals for these printers:

**C. ITOH, GUME, CITIZEN,
OKIDATA, FUJITSU, EPSON
AND OTHERS**

For Visa/MC/Amex Call
1-800-524-8338

Precision Images
P.O. Box 573
Chester, NY 10918

Circle Reader Service Number 195

Copies of articles from this publication are now available from the UMI Article Clearinghouse.

Mail to: University Microfilms International
300 North Zeeb Road, Box 91 Ann Arbor, MI 48106

G

COMPUTE!'s SpeedScript Disk

**ONLY
\$9.95!**

A powerful word processing package for Commodore and Apple II owners

A Great Deal for Commodore Users!

- SpeedScript for the 64
- SpeedScript 128—80-column version
- Spelling checkers
- Plus more than a dozen other SpeedScript support utilities all on one disk (including full documentation)

Two Versions for Apple Users!

- SpeedScript 3.0—40-column word processor
- SpeedScript 80—80-column version
- Conversion programs to move files from ProDOS to DOS 3.3 and back
- Full documentation on disk

YES! Send me _____ copies of COMPUTE!'s SpeedScript Disk.

I've enclosed \$9.95 plus \$2.00 postage and handling. (Outside U.S. and Canada add \$1.00 for surface mail or \$3.00 for air mail.)

ORDER NOW!

_____ Amount
_____ Sales Tax*
_____ Total

Name _____

Address _____

City _____ State _____ ZIP _____

Check here for: ☐ Apple version
☐ Commodore version

Mail personal check or money order to

Commodore SpeedScript Disk or Apple SpeedScript Disk
P.O. Box 5188 P.O. Box 5188
Greensboro, NC 27403 Greensboro, NC 27403

*Residents of New York, Pennsylvania, and North Carolina must add appropriate sales tax. All orders must be paid in U.S. funds by a check drawn on a U.S. bank. Sorry, no credit card orders accepted.

Please allow 4-6 weeks for delivery. Program available only on 5 1/4-inch disks.